Dictionary

- A datatype stores a sequence of pairs of key and value.
- Syntax: { key1 : value1 , key2 : value2 , key3 : value3 }
  pair #1 pair #2 pair #3
- Access by key: you can access the value of that key by indexing with the key.
  Syntax: dict[key1]
  Value 1
- Key must be an immutable object
  → string, tuple, int, float
- Value can be anything → string, list, tuple, dict, ...

EX1

```
my_dict = {"Alice":21, "Bob":32, "Chris":50}
print(my_dict["Alice"])
X = my_dict["Bob"] + my_dict["Chris"]
print(X)
print(my_dict)
```

21

32 + 50 = 82

{'Alice':21, 'Bob':32, 'Chris':50}

Update / Modify existing pairs

Syntax: dict[key] = new_value

assign new value to this pair's value

Syntax: dict[key] = new_value

assign new_value to this pair's value (key's)

```
my_dict = {'A': 5, 'B': 6, 'C': 7}
print(my_dict['B'])
my_dict['B'] = 10
print(my_dict['B'])
print(my_dict)
```

→ 6 (6 → 10)

→ 10

→ {'A': 5, 'B': 10, 'C': 7}

Cannot access a key that doesn't exist in the dict

```
dict1 = {'A': 3, 'B': 5}
print(dict1['C'])
```

'C' not in dict1!! → Error!

Cannot access!!

- **in operator**
  - ↳ check whether a key is in the dict
    - ↳ True if in, False otherwise

```
('A' in dict1) → True
('C' in dict1) → False
```

- **len() function**
  - ↳ return a number of pairs

```
len(dict1) → 2
```

Adding new pairs to dict

Syntax: `dict[new_key] = new_value`

new pair → new_key: new_value

new pair → new_key : new_value

Ex new_dict = {} ← empty dict

new_dict['Alice'] = 22 → 'Alice': 22
new_dict['Bob'] = 40 'Bob': 40
print(new_dict) → {'Alice': 22, 'Bob': 40}

new_dict['Chris'] = 11
print(new_dict) → {'Alice': 22, 'Bob': 60, 'Chris': 11}

Ex2 Creating a dict from 2 lists

names = ["Alice", "Bob", "Chris"]
scores = [30, 25, 15]

→ {'Alice': 30, 'Bob': 25, 'Chris': 15}

Alice got 30.

```
new_dict = {}
for i in range(len(names)):
    new_dict[names[i]] = scores[i]
```

Brute-force

```
new_dict = {}
new_dict[names[0]] = scores[0]
new_dict[names[1]] = scores[1]
new_dict[names[2]] = scores[2]
```

('Alice' 30)

Looping through a dictionary

- for loop will loop through keys in the dict
  - ↳ the variable will store a key in order!

```
for key in dict
```

Ex3

```
my_dict = {'Alice': 10, 'Bob': 12, 'Chris': 13}
for x in my_dict:
    print(x, my_dict[x])
```

(1) (2) (3)

→ Alice 10
Bob 12
Chris 13

---

print(x, my_dict[x]) Bob 12
Chris 13

Useful dictionary Methods

Syntax: dict.method_name()

.keys() → returns all keys as a 'list'

.values() → returns all values as a 'list'

.items() → returns all pairs as a 'list' of tuple of (key, value)

Ex
```
mydict = {'Alice': 20, 'Bob': 30}
mydict.keys()   → ['Alice', 'Bob']
mydict.values() → [20, 30]
mydict.items()  → [('Alice', 20), ('Bob', 30)]
```

* Always use these methods with for loop!!

Ex
```
for key, value in mydict.items():
    print(key, value)
```
('Alice', 20)

k, v

name, score

→ Alice 20
Bob 30

.get(key, default)

↳ it will return the value of that key if the key exists in the dict.

Otherwise, it will return default.

```
def ourget(dict, key, default):
    if key in dict:
        return dict[key]
    else:
        return default
```

```
my_dict = {'A': 1, 'B': 2, 'C': 3}
print(my_dict.get('A', -1))   → 1
print(my_dict.get('X', -1))   → -1
```

'X' not in my_dict!

Counting things with dictionary

- Count how many apple, banana, orange in a list

```
fruits = ['apple', 'banana', 'apple', 'orange', 'apple']

apple_n = 0
banana_n = 0
orange_n = 0
for fruit in fruits:
    if fruit == 'apple':
        apple_n += 1
    elif fruit == 'banana':
        banana_n += 1
    elif fruit == 'orange':
        orange_n += 1
print(apple_n, banana_n, orange_n)
```

idea: key → fruit
value → count

```
d_fruit = {"apple": 0,
           "banana": 0,
           "orange": 0}
for fruit in fruits:
    if fruit in d_fruit:
        d_fruit[fruit] += 1
print(d_fruit)
```

check fruit in d_fruit so that you can access

d_fruit[fruit] = d_fruit[fruit] + 1

access!

Counting words in a sentence

```
sentence = "A cat likes a dog A cat A dog"
```

↳ 4x 'a', 2x 'cat', 2x 'dog', 1x 'like'

```
sentence = sentence.lower()
word_list = sentence.split(' ')
```

↳ ['a', 'cat', 'likes', 'a', 'dog', 'a', ...]

```
d_word = {}
for word in word_list:
```

```
d_word = {}
for word in word_list:
    if word in d_word:
        d_word[word] += 1
    else:  (first time encounter the word not in the dict yet)
        d_word[word] = 0 + 1
print(d_word)
```

d_word[word] = d_word.get(word, 0) + 1

default

Nested dictionary

Dict of dict

```
users = { "Alice" : {"age": 25, "gender": 'F'}
          "Bob"   : {"age": 30, "gender": 'M'}
          "Chris" : {"age": 77, "gender": 'M'} }

print(users['Alice'])  → {'age': 25, 'gender': 'F'}
print(user['Alice']['age'])  → 25
print(user['Chris']['gender'])  → M
```

Dict of list

```
my_dict = { "Alice" : [1, 2, 3]
            "Bob"   : [4, 5, 6] }

print(my_dict['Alice'])  → [1, 2, 3]
print(my_dict['Alice'][1])  → 2
print(sum(my_dict['Alice']))  → 6
```

```
print(sum(my_dict['Alice']))
my_dict['Alice'].append(10)
print(my_dict)
```

sum([1,2,3]) → 6

print(sum(my_dict['Alice'])) → 6

print(my_dict) → {'Alice': [1, 2, 3, 10], 'Bob': [4, 5, 6]}

Dict: {key1: value1, key2: value2}

Access by key: dict[key1] → value1

Update/modify/add: dict[key] = value

for loop through key

Methods: .keys(), .values(), .items(), .get(key, default)

Logic: counting things with a dict